00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D

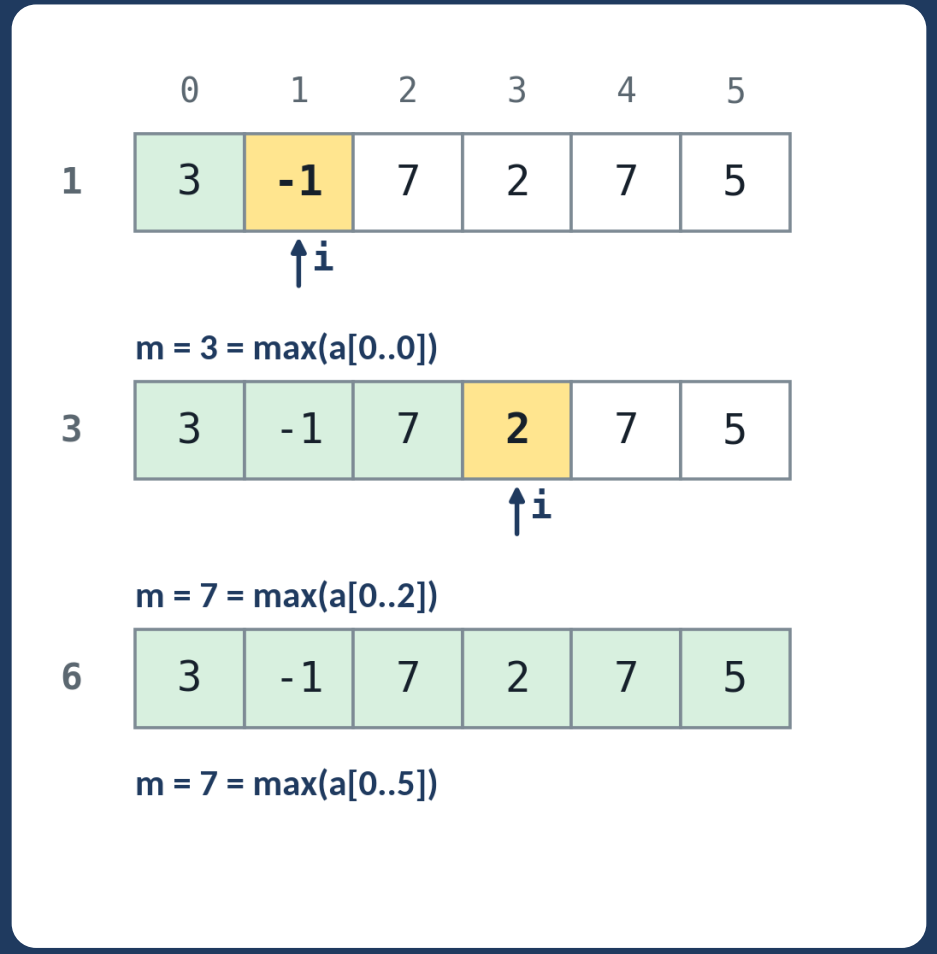
ALGORİTMA VE PROGRAMLAMA – II

HAFTA 01

Algoritmalara Dönüş: Doğruluk, Döngü Değişmezleri ve Deneysel Ölçüm

Sözleşme · Hoare üçlüsü · döngü değişmezi · sonlanma · kâhinle test · tekrarlanabilir ölçüm · C · C++ · Python

MODÜL I · ALGORİTMALARIN DİLİ



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 **Doğruluk ve Ölçüm**	**H2** Asimptotik Analiz	**H3** Özyineleme ve Geri İzleme	**H4** Arama Algoritmaları	**H5** Temel Sıralama	**H6** Böl ve Fethet	**H7** Öbek ve Doğrusal Sıralama
0x00	0x01	0x02	0x03	0x04	0x05	0x06
H8 Dinamik Dizi ve Bağlı Liste	**H9** Yığın ve Kuyruk	**H10** Karma Tabloları	**H11** Graflar: BFS ve DFS	**H12** Açgözlü Algoritmalar	**H13** Dinamik Programlama	**H14** Sentez ve Final Projesi
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14

TAHMİN ET

AP-1’den: Bu program ne yazdırır?

quiz/H01_Q01_dizi_adim.c · satır 8–17 — C

```c
int main(void)
{
    int a[] = {3, 1, 4, 1, 5};
    int t = 0;
    for (int i = 0; i < 5; i += 2) {
        t += a[i];
    }
    printf("%d\n", t);
    return 0;
}
```

A 9

B 12

C 14

D 3

AP-1’den: değerle aktarım ve işaretçi

S2 · main’deki x kaç?

quiz/H01_Q02_deger_aktarim.c · satır 8–20 C

```c
void arttir(int x)
{
    x = x + 1;
    printf("içeride x = %d\n", x);
}

int main(void)
{
    int x = 5;
    arttir(x);
    printf("main'de x = %d\n", x);
    return 0;
}
```

S3 · ekle(&x, 4) sonrası x?

quiz/H01_Q03_isaretci.c · satır 8–19 C

```c
void ekle(int *p, int k)
{
    *p += k;
}

int main(void)
{
    int x = 3;
    ekle(&x, 4);
    printf("%d\n", x);
    return 0;
}
```

Yanıtlar: gerçek çıktılar

S1 · H01_Q01_dizi_adim

ÇIKTI

```
$ ./H01_Q01_dizi_adim
12
```

i = 0, 2, 4 → 3 + 4 + 5

S2 · H01_Q02_deger_aktarim

ÇIKTI

```
$ ./H01_Q02_deger_aktarim
içeride x = 6
main'de x = 5
```

C'de argüman değerle aktarılır: fonksiyon kopyayı değiştirir

S3 · H01_Q03_isaretci

ÇIKTI

```
$ ./H01_Q03_isaretci
7
```

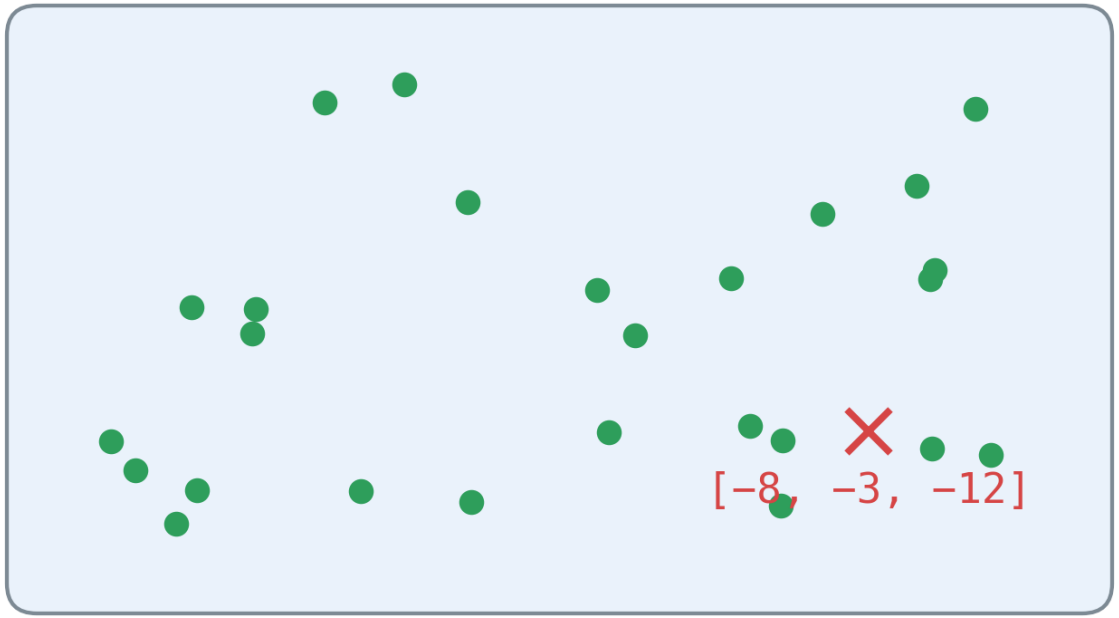
Adres aktarılır: *p += k çağıranın x'ine yazar

Bu hafta: bir fonksiyonun **ne aldığı, neyi değiştirebileceği ve ne döndüreceği** yazılı bir **sözleşmedir**.

HAFTANIN BÜYÜK SORUSU

Bir programın bütün testleri geçmesi, o programın doğru olduğunu kanıtlar mı?

Test: seçilmiş girdiler



geçen testler ● gözden kaçan hata ✗

Kanıt: değişmez + sonlanma

```
m = max(a[0..i−1])
```

başlatma · koruma · sonlanma

her geçerli girdi için

Test seçilmiş girdileri dener; değişmez ve sonlanma kanıtı her geçerli girdiyi kapsar

GERÇEK DÜNYADAN

2015

Timsort: yıllarca testlerden geçen kodda değişmez ihlali

- Python'un ve Java'nın (nesne dizileri) standart sıralaması; koşu uzunluklarını bir yığında tutar ve bir **eşitsizlik zincirinin** (değişmez) korunduğunu varsayar.
- de Gouw ve arkadaşları OpenJDK gerçekleştirimini **KeY** ile biçimsel olarak doğrulamaya çalışırken değişmezin her durumda korunmadığını buldu: kurgulanmış girdide `ArrayIndexOutOfBoundsException`.
- Aynı kusur Python'da da vardı (pratikte erişilemeyecek büyüklükte girdi gerekiyordu); ikisi de düzeltildi.
- Dijkstra: test hataların **varlığını** gösterebilir, **yokluğunu** kanıtlayamaz (Dahl vd., 1972).

de Gouw vd., 2015 (CAV 2015); Dahl vd., 1972

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 1.1	Bir hesaplama problemini girdi kümesi, çıktı koşulu ve örnek (instance) kavramlarıyla tanımlar; algoritmayı ön koşul–son koşul sözleşmesiyle ifade eder.
ÖÇ 1.2	Döngü değişmezini başlatma–koruma–sonlanma adımlarıyla kurar ve basit algoritmaların (toplam, en büyük eleman, doğrusal arama, Öklid) doğruluğunu gerekçelendirir.
ÖÇ 1.3	assert, sınır değer testleri ve kaba kuvvet bir kâhinle (test oracle) rastgele karşılaştırmalı testler yazar.
ÖÇ 1.4	Çalışma süresini C, C++ ve Python'da ölçer; tekrar, ısınma ve ortanca (medyan) kullanarak ölçüm gürültüsünü azaltan bir deney düzeni kurar.
ÖÇ 1.5	Aynı algoritmanın C, C++ ve Python karşılıklarını okur; üç dilde sözdizimi benzer ama anlamı farklı yapıları (tamsayı bölmesi, kalan, taşma) tanır.

Hoare üçlüsü: {P} S {Q}

P · ön koşul	S · program	Q · son koşul
`{ x == 5 }`	`x = x + 1;`	`{ x == 6 }`
`{ x ≥ 0 ∧ y > 0 }`	`bol(x, y, &q, &r)`	`{ x == r + y·q ∧ 0 ≤ r < y }`
`{ n ≥ 1 }`	`m = en_buyuk(a, n)`	`{ m ∈ a ∧ ∀k: a[k] ≤ m }`

Kısmi doğruluk: P'yi sağlayan bir durumda başlayan S SONLANIRSA Q sağlanır.

Tam doğruluk = kısmi doğruluk + sonlanma · Hoare üçlüsü (Hoare, 1969)

P doğruyken başlayan S sonlanırsa Q doğrudur (Hoare, 1969); sonlanma ayrıca gösterilir

KOD

Hoare’nin bölme örneği: değişmez x = r + y·q

kavram/H01_K01_hoare_bolme.c · satır 13–24

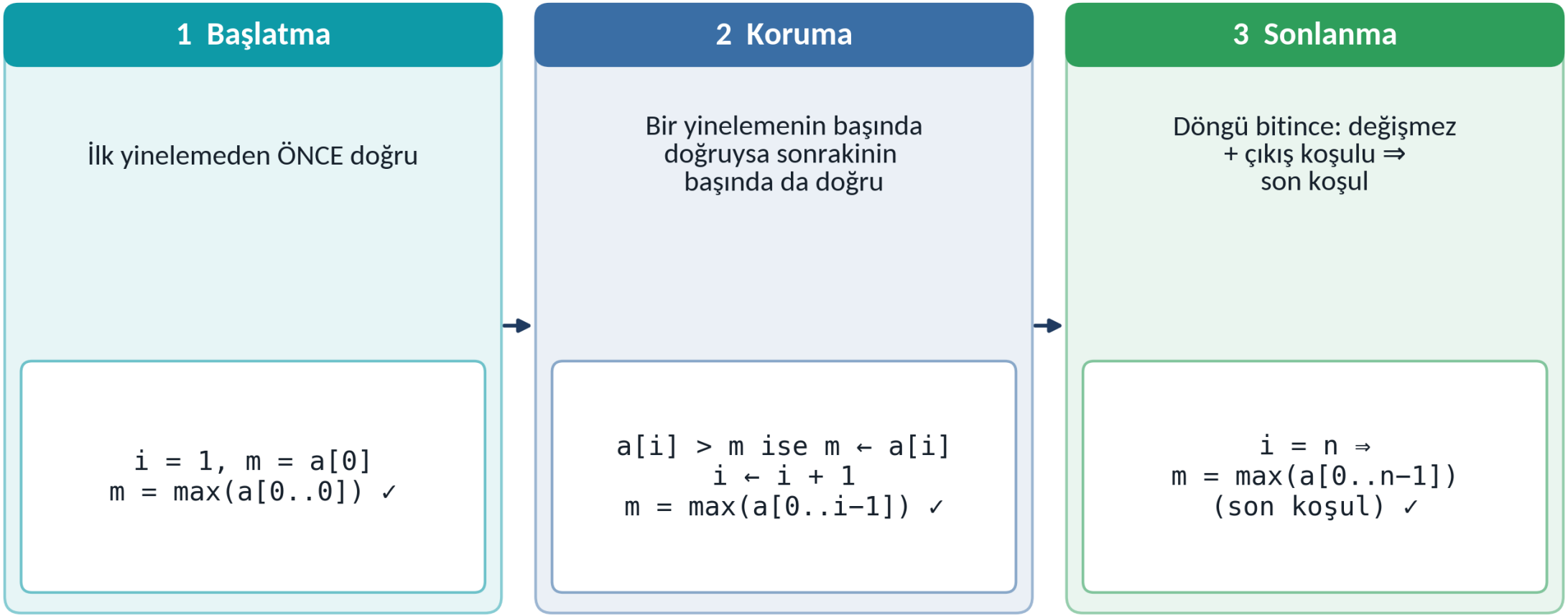
C

```c
void bol(int x, int y, int *q, int *r)
{
    assert(x >= 0 && y > 0);            /* P: ön koşul              */
    *r = x;
    *q = 0;
    while (y <= *r) {
        assert(x == *r + y * *q);       /* değişmez (döngü başı)  */
        *r = *r - y;
        *q = *q + 1;
    }
    assert(x == *r + y * *q && 0 <= *r && *r < y);   /* Q: son koşul */
}
```

ÇIKTI

```
$ ./H01_K01_hoare_bolme
17 = 5 * 3 + 2
 4 = 9 * 0 + 4
```

Döngü değişmezi: başlatma, koruma, sonlanma



Tümevarıma benzer: başlatma = taban adımı, koruma = tümevarım adımı; sonlanma döngüye özgüdür

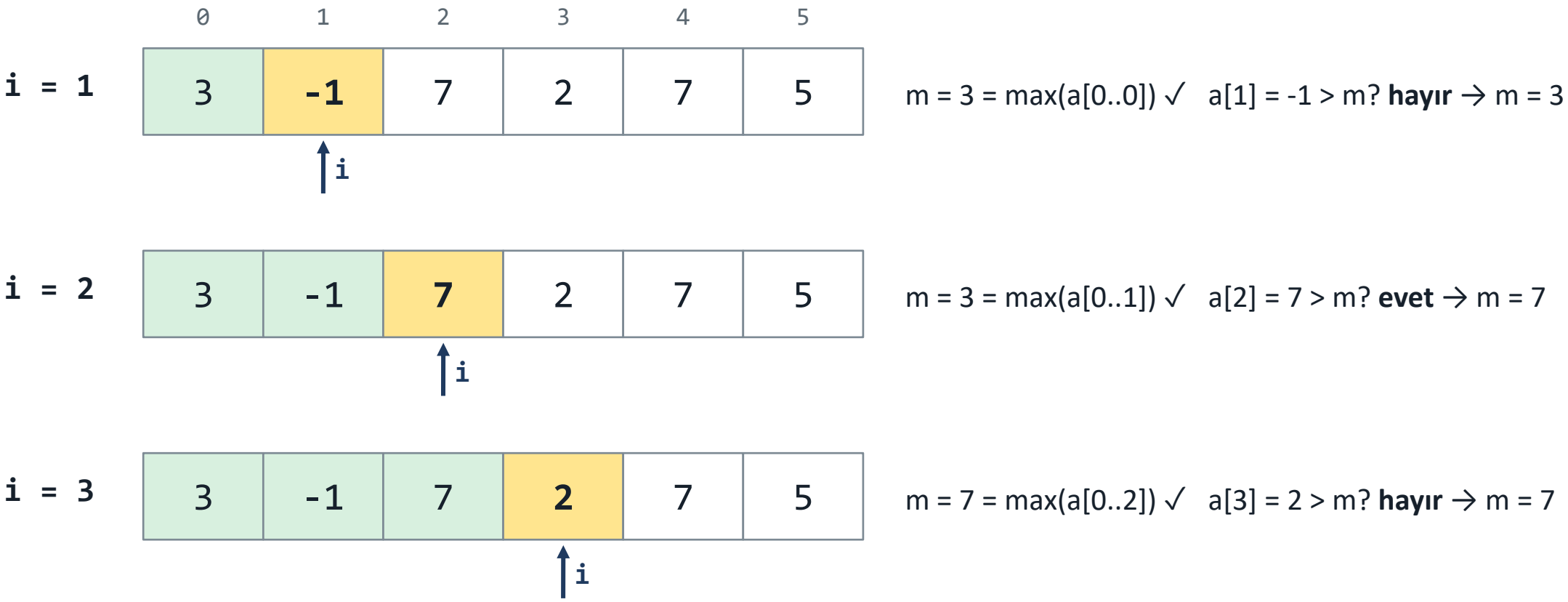
Başlatma = taban adımı, koruma = tümevarım adımı; sonlanma tümevarımı döngüye bağlar (Cormen vd., 2022)

01

Üç amiral algoritma

En büyük eleman · doğrusal arama · Öklid: problem → fikir → izleme → değişmez → kimlik kartı

Elle izleme (1/2): her satır bir yinelemenin başı



TAHMİN ET

Sizce sırada ne olacak? i = 4’te a[4] = 7 ve m = 7

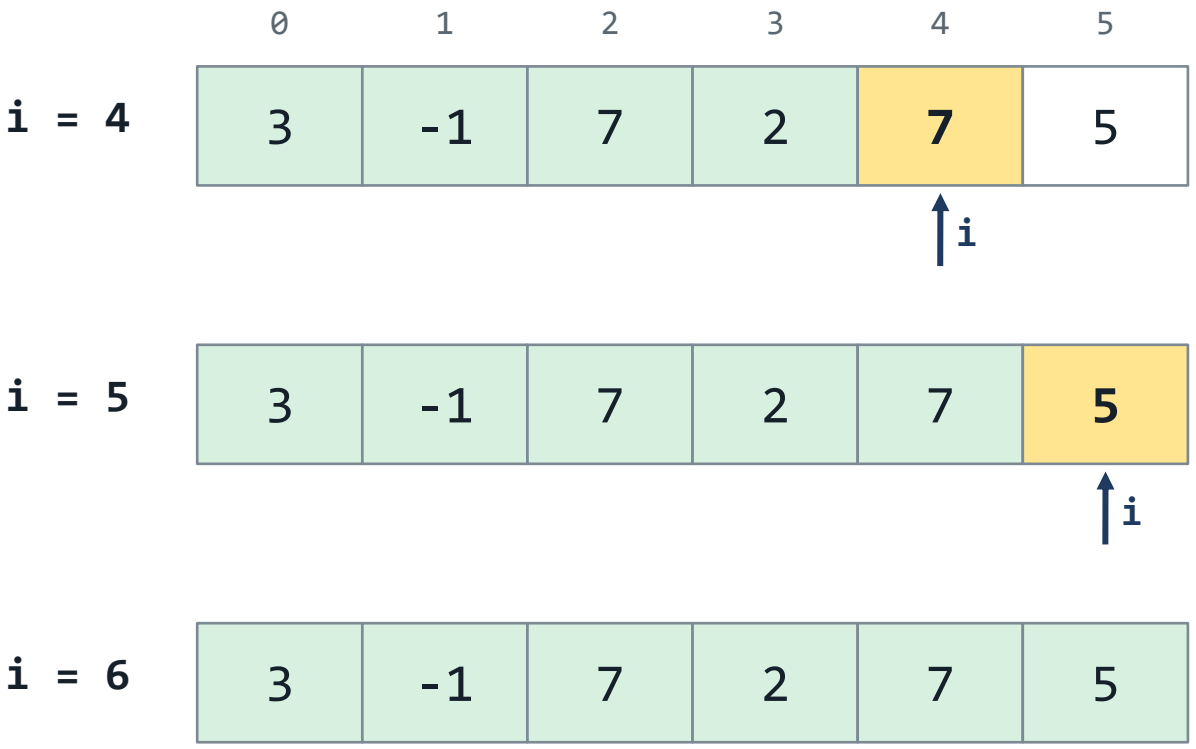
A döngü erken durur

B m yeniden 7 olur, en büyüğün konumu 4’e kayar

C m = 7 kalır: 7 > 7 yanlış

D değişmez bozulur

Elle izleme (2/2): sonlanma



m = 7 = max(a[0..3]) ✓ a[4] = 7 > m? **hayır** → m = 7

m = 7 = max(a[0..4]) ✓ a[5] = 5 > m? **hayır** → m = 7

m = 7 = max(a[0..5]) ✓ **i = n: döngü biter**

karşılaştırma: 5 = n − 1

EN-BÜYÜK: sözde kod ve değişmez

```
1  algoritma EN-BÜYÜK(a[0..n−1])      ▷ ön koşul: n ≥ 1
2      m ← a[0]
3      i ← 1'den n−1'e kadar her i için
4          ▷ değişmez: m = max(a[0..i−1])
5          eğer a[i] > m ise
6              m ← a[i]
7      döndür m          ▷ son koşul: m = max(a[0..n−1])
```

- **Başlatma:** i = 1, m = a[0] = max(a[0..0])
- **Koruma:** a[i] > m ise m ← a[i]; değilse m ≥ a[i]: gövde sonunda m = max(a[0..i])
- **Sonlanma:** i = n → m = max(a[0..n−1])
- **Sonlanma kanıtı:** n − i her adımda 1 azalır, 0'ın altına inmez
- **Değişmezi bulmak:** son koşulu gevşet: n yerine i → m = max(a[0..i−1])

C gerçekleştirimi: en_buyuk

H01_01_en_buyuk.c · satır 19–28

```c
int en_buyuk(const int a[], size_t n)
{
    int m = a[0];
    for (size_t i = 1; i < n; i++) {
        if (a[i] > m) {
            m = a[i];
        }
    }
    return m;
}
```

ÇIKTI

```
$ ./H01_01_en_buyuk
a = [3, -1, 7, 2, 7, 5] -> en büyük: 7
b = [-8, -3, -12] -> en büyük: -3
c = [42] -> en büyük: 42
```

- `const int a[]`: diziyi değiştirmeme sözü
- n = 0 ön koşul ihlalidir: `a[0]`'ı okumak tanımsız davranış
- Üç girdi: karışık, **tamamı negatif**, **tek eleman**

KOD

Değişmezi assert ile denetlemek (Örnek 2)

H01_02_en_buyuk_assert.c · satır 38–52

C

```c
int en_buyuk(const int a[], size_t n)
{
    assert(n >= 1);                    /* ön koşul */
    int m = a[0];
    size_t i = 1;
    assert(degismez(a, i, m));         /* başlatma */
    while (i < n) {
        if (a[i] > m) {
            m = a[i];
        }
        i++;
        assert(degismez(a, i, m));     /* koruma */
    }
    assert(i == n);                    /* sonlanma: i == n */
    return m;  /* değişmez + (i == n)  =>  m = max(a[0..n-1]) */
```

ÇIKTI

```
$ ./H01_02_en_buyuk_assert
Değişmez assert ile 10 kez denetlendi (başarısız olsaydı program dururdu).
$ (-DNDEBUG ile)
Değişmez assert ile 0 kez denetlendi (başarısız olsaydı program dururdu).
```

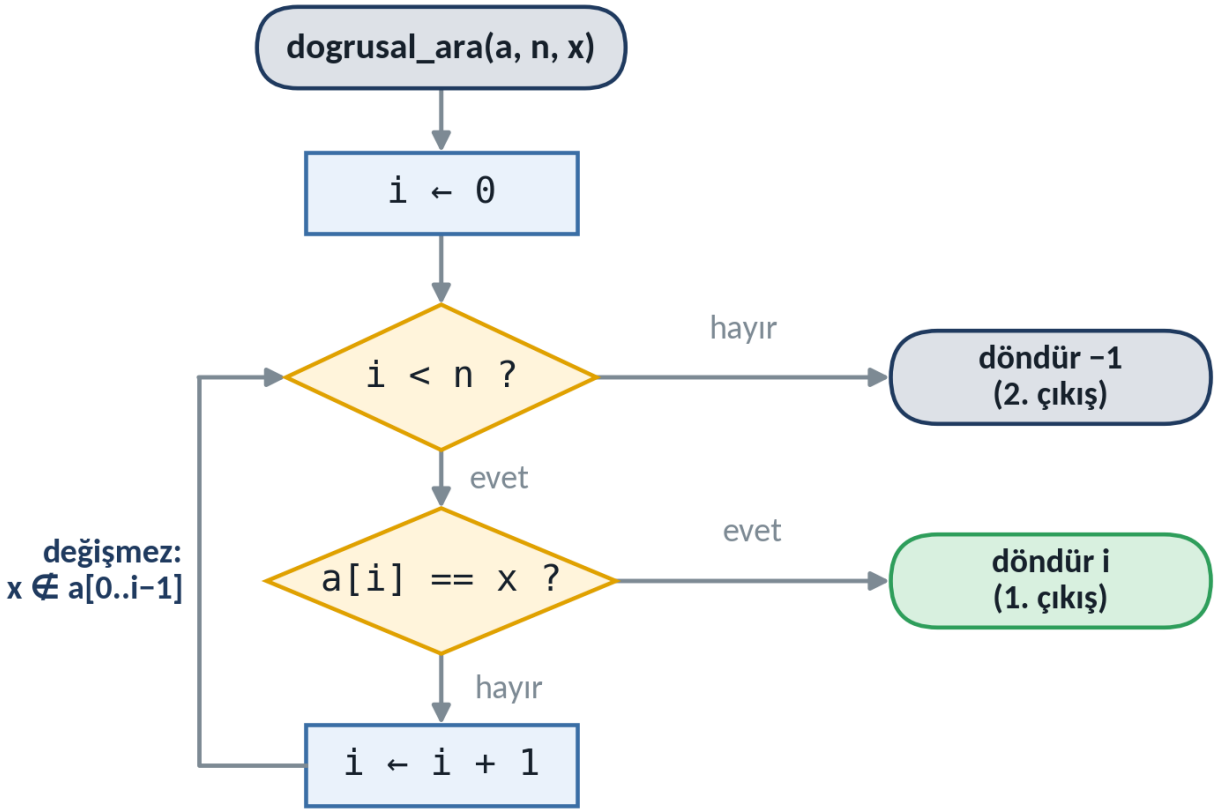
ALGORİTMA KİMLİK KARTI

En büyük eleman *maximum element, linear scan*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Kaba kuvvet (tek geçiş)	Dizi boş değil (n ≥ 1); sıralı olması gerekmez	Θ(n)	Θ(n)	Θ(n)	O(1)

Özellikler	Her girdide tam n − 1 karşılaştırma; diziyi değiştirmez; başlangıç değeri a[0] olmalı (0 değil: tamamı negatif dizide hata)
Değişmez / doğruluk	Döngü başında m = max(a[0..i−1]); i = n'de m = max(a[0..n−1])
Kütüphane karşılığı	C: yok (elle yazılır) · C++: `*std::max_element(a.begin(), a.end())` · Python: `max(a)`
Ne zaman kullanılır	Sırasız veride en büyük (ya da < ile en küçük) değer bir kez aranacaksa
Ne zaman kullanılmaz	En büyük sürekli değişen veride sık sık sorulacaksa (öbek / öncelik kuyruğu, H7); dizi zaten sıralıysa son eleman O(1)

Problem ve akış: ilk konum ya da “bulunamadı”



- **Girdi:** a[0..n−1] (n = 0 olabilir), x
- **Çıktı:** x varsa **ilk** konum i; yoksa −1 (Python: None)
- **Değişmez:** döngü başında x ∉ a[0..i−1] (“şimdiye kadar bakılanların hiçbiri x değil”)
- Analoji: raftaki kitapları soldan sağa okumak; bulunca dur
- Çağıran “bulunamadı”yı **denetlemek zorunda**

sonlanma: n − i her yinelemede 1 azalır; en çok n yinelemeden sonra i = n

Elle izleme: x = 3 (bulundu) ve x = 6 (yok)



x = 3: a[0] = 7 == x? hayır · değişmez: elenmiş bölge boş

a[1] = 3 == x? **evet** → döndür 1 (2 karşılaştırma)

x = 6: i = n = 7 → döndür −1 (7 karşılaştırma)

elenmiş: x orada yok

etkin: a[i]

bulundu

TAHMİN ET

Sizce? Aynı dizide x = 7 ve x = 8 aranırsa kaç karşılaştırma yapılır?

A 1 ve 8

B 7 ve 7

C 0 ve 6

D 1 ve 7

DOĞRUSAL-ARA: iki çıkış, iki gerekçe

```
1  algoritma DOĞRUSAL-ARA(a[0..n−1], x)
2      i ← 0'dan n−1'e kadar her i için
3          ▷ değişmez: x ∉ a[0..i−1]
4          eğer a[i] = x ise
5              döndür i          ▷ 1. çıkış: ilk konum
6      döndür −1                 ▷ 2. çıkış: x ∉ a[0..n−1]
```

- **Başlatma:** i = 0, bölüm boş
- **Koruma:** a[i] ≠ x ise x ∉ a[0..i]
- **1. çıkış:** a[i] = x ve x ∉ a[0..i−1] → i **ilk** konum
- **2. çıkış:** i = n → x ∉ a[0..n−1]
- **Sonlanma:** n − i azalır; en çok n yineleme

KOD

C gerçekleştirimi: dogrusal_ara

H01_04_dogrusal_arama.c · satır 22–32

C

```c
int dogrusal_ara(const int a[], size_t n,
                 int x)
{
    for (size_t i = 0; i < n; i++) {
        karsilastirma++;
        if (a[i] == x) {    /* 1. çıkış */
            return (int)i;
        }
    }
    return -1;              /* 2. çıkış */
}
```

ÇIKTI

```
$ ./H01_04_dogrusal_arama
a = [7, 3, 9, 3, 5, 1, 8] (n = 7)
ara(7) -> 0, karşılaştırma: 1
ara(3) -> 1, karşılaştırma: 2
ara(8) -> 6, karşılaştırma: 7
ara(6) -> bulunamadı, karşılaştırma: 7
boş dizi:
ara(7) -> bulunamadı, karşılaştırma: 0
```

ALGORİTMA KİMLİK KARTI

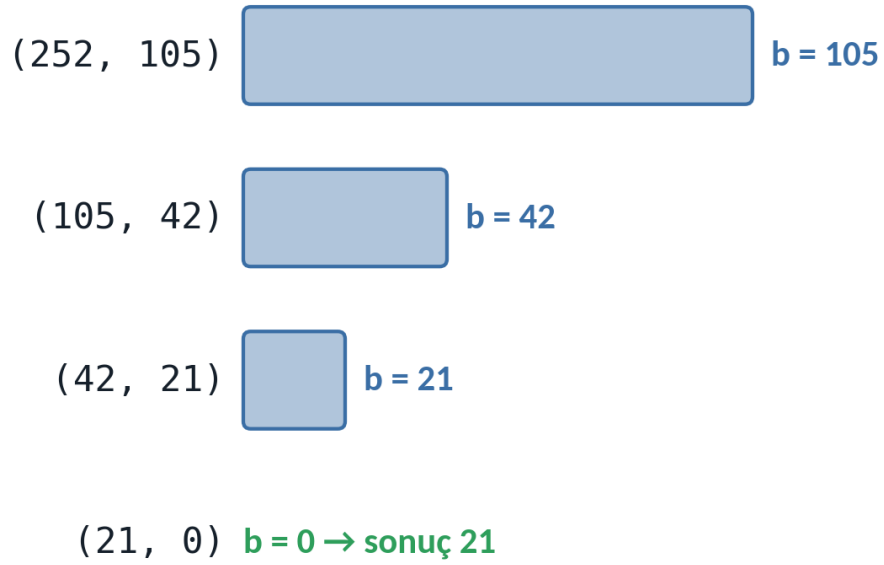
Doğrusal arama *linear search*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Kaba kuvvet (tek geçiş, erken çıkış)	Yok — dizi sıralı olmak zorunda değil; boş dizi de geçerli	O(1)	Θ(n)	Θ(n)	O(1)

Özellikler	İki çıkış noktası: ilk konumu döndürür; yoksa −1 (C/C++) ya da None (Python); aranan yoksa n karşılaştırma
Değişmez / doğruluk	Döngü başında aranan a[0..i−1] içinde yok
Kütüphane karşılığı	C: standart C'de yok (POSIX `lfind`) · C++: `std::find` · Python: `a.index(x)` (yoksa `ValueError`), `x in a`
Ne zaman kullanılır	Küçük ya da sırasız veride, tek seferlik aramada; sıralama maliyetine değmeyecek durumlarda
Ne zaman kullanılmaz	Aynı veride çok sayıda arama yapılacaksa: önce sıralayıp ikili arama (H4) ya da karma tablosu (H10)

Fikir: ebob(a, b) = ebob(b, a mod b)

ebob(252, 105): (a, b) ← (b, a mod b)



b kesin azalır: 0 ≤ a mod b < b

değişmez: ebob(a, b) hep 21

- a ve b'yi bölen her sayı a − q·b'yi de böler; tersi de doğru
- Problem her adımda **küçülür**: b yerine 0 ≤ a mod b < b
- Analoji: 252 × 105 dikdörtgeni en büyük kare fayansla döşemek
- b = 0 olunca dur: ebob(a, 0) = a

İzleme: ebob(252, 105) = 21

H01_05_oklid_ebob.c · satır 22–32

```c
int ebob(int a, int b)
{
    adim_sayisi = 0;
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
        adim_sayisi++;
    }
    return a;
}
```

a	b	a mod b
252	105	42
105	42	21
42	21	0
21	0	b = 0 → 21

TAHMİN ET

Sizce? ebob(55, 89) çağrısında a < b. İlk adımda ne olur?

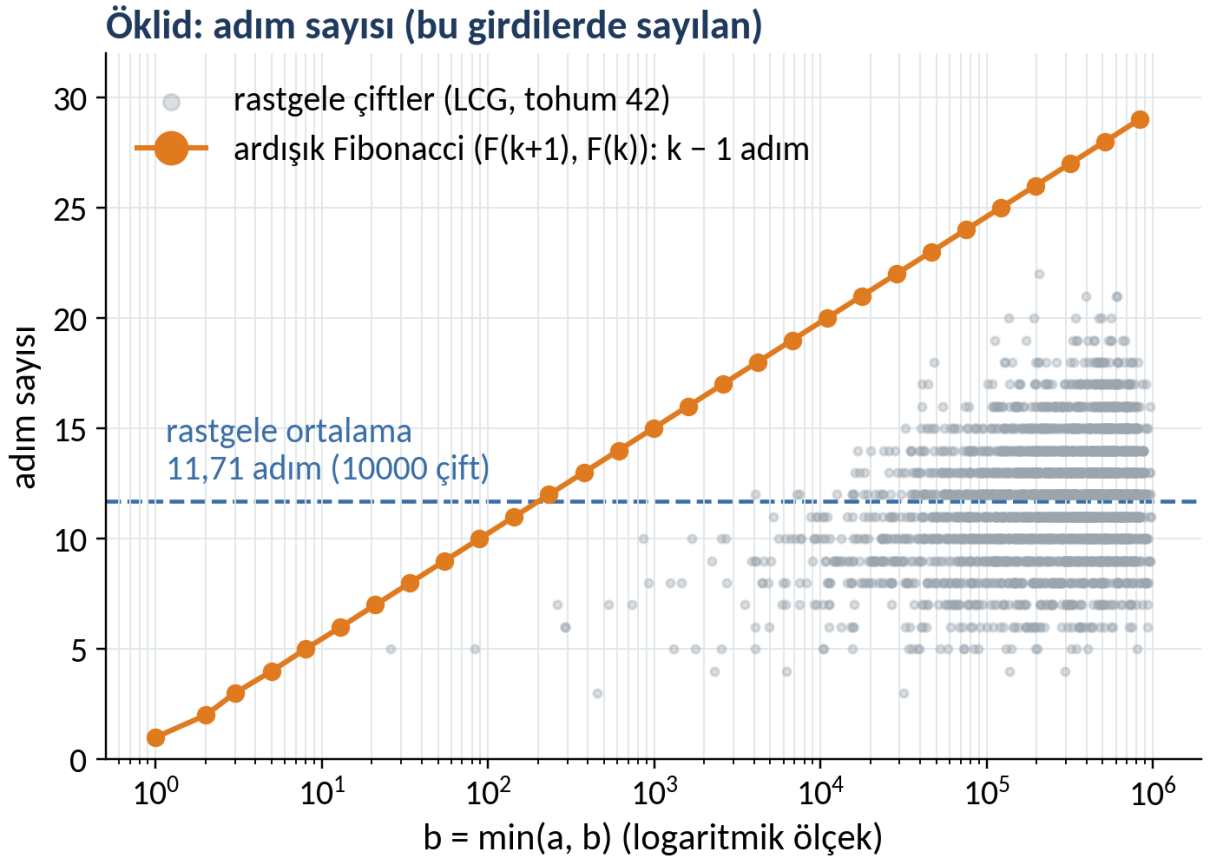
A 55 mod 89 = 55: a ile b yer değiştirir

B Hata: a ≥ b olmalı

C Sonuç hemen 55

D Döngü sonsuza dek döner

En kötü durum: ardışık Fibonacci sayıları



- (F(k+1), F(k)): her adım bir önceki çifte iner → k − 1 adım (Lamé; Cormen vd., 2022)
- Programla: (89, 55) → 9; (10 946, 6 765) → 19; (832 040, 514 229) → 28
- 10 000 rastgele çift (LCG): ortalama 11,71, en çok 22 adım
- Adım sayısı **O(log min(a, b))**

ALGORİTMA KİMLİK KARTI

Öklid algoritması (EBOB) *Euclidean algorithm, GCD*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Problemi küçültme: ebob(a, b) = ebob(b, a mod b)	a ≥ 0, b ≥ 0 tamsayı; ikisi birden 0 değil	O(1)	O(log min(a, b))	O(log min(a, b))	O(1)

Özellikler	Sonlanma: b her adımda kesin azalır, 0'ın altına inemez; en kötü durum ardışık Fibonacci sayıları, ör. (832040, 514229) için 28 adım
Değişmez / doğruluk	ebob(a, b) döngü boyunca aynı kalır, çünkü ebob(a, b) = ebob(b, a mod b)
Kütüphane karşılığı	C: yok (elle yazılır) · C++: `std::gcd` (`<numeric>`, C++17) · Python: `math.gcd(a, b)`
Ne zaman kullanılır	İki tamsayının EBOB'u, kesir sadeleştirme, EKOK = a / ebob(a, b) · b
Ne zaman kullanılmaz	Asal çarpanlar gerekiyorsa (ayrı problem); negatif girdide önce mutlak değer alın (C'de % negatif kalan verebilir, Örnek 6)

Üç dilli çalışma düzeni

C | C++ | **Python** | H01_04_dogrusal_arama.py | Farkları göster ✓

```python
def dogrusal_ara(a, x):
    """x'in ilk konumu; yoksa None."""
    global karsilastirma
    for i in range(len(a)):
        karsilastirma += 1
        if a[i] == x:    # 1. çıkış
            return i
    return None          # 2. çıkış
```

DİL PENCERESİ

"Bulunamadı": C/C++'ta −1, Python'da None (is None ile sınanır); sayaca yazmak için global

- **Üç dosya, aynı ad:** `c/`, `cpp/`, `python/` altında `HNN_SS_ad`
- **Derleme:** `-O2` ile; C17 ve C++17 standardı, uyarı bayrakları açık ve uyarısız; Python 3.10+ (komutlar: ders notu 3.10)
- **Eşdeğerlik:** `python3 esdeger_test.py .` → "Toplam 13 örnek · geçen 13 · hatalı 0"
- **=** üç dilde aynı çıktı; **Δ** bilinçli dil farkı (Örnek 6, 7)
- Web: C | C++ | Python sekmeleri; **farkları göster** hardal satırları açar

Üç dilde: en_buyuk

H01_01_en_buyuk.c · satır 19–28 — C

```c
int en_buyuk(const int a[], size_t n)
{
    int m = a[0];
    for (size_t i = 1; i < n; i++) {
        if (a[i] > m) {
            m = a[i];
        }
    }
    return m;
}
```

H01_01_en_buyuk.cpp · satır 21–31 — C++

```cpp
int en_buyuk(const std::vector<int>& a)
{
    const std::size_t n = a.size();
    int m = a[0];
    for (std::size_t i = 1; i < n; i++) {
        if (a[i] > m) {
            m = a[i];
        }
    }
    return m;
}
```

H01_01_en_buyuk.py · satır 16–22 — Python

```python
def en_buyuk(a):
    """a'nın en büyük elemanı."""
    m = a[0]
    for i in range(1, len(a)):
        if a[i] > m:
            m = a[i]
    return m
```

C: dizi + boyut · C++: `const std::vector<int>&` · Python: liste boyutunu taşır

hardal satır = C'den farklılaşan satır

Üç dilde: dogrusal_ara

H01_04_dogrusal_arama.c · satır 22–32 — C

```c
int dogrusal_ara(const int a[], size_t n,
                 int x)
{
    for (size_t i = 0; i < n; i++) {
        karsilastirma++;
        if (a[i] == x) {    /* 1. çıkış */
            return (int)i;
        }
    }
    return -1;              /* 2. çıkış */
}
```

H01_04_dogrusal_arama.cpp · satır 23–34 — C++

```cpp
int dogrusal_ara(const std::vector<int>& a,
                 int x)
{
    const std::size_t n = a.size();
    for (std::size_t i = 0; i < n; i++) {
        karsilastirma++;
        if (a[i] == x) {    // 1. çıkış
            return static_cast<int>(i);
        }
    }
    return -1;              // 2. çıkış
}
```

H01_04_dogrusal_arama.py · satır 20–27 — Python

```python
def dogrusal_ara(a, x):
    """x'in ilk konumu; yoksa None."""
    global karsilastirma
    for i in range(len(a)):
        karsilastirma += 1
        if a[i] == x:    # 1. çıkış
            return i
    return None          # 2. çıkış
```

"Bulunamadı": C ve C++'ta `-1`, Python'da `None` (`is None` ile sınanır)

hardal satır = C'den farklılaşan satır

Üç dilde: ebob

H01_05_oklid_ebob.c · satır 22–32 — C

```c
int ebob(int a, int b)
{
    adim_sayisi = 0;
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
        adim_sayisi++;
    }
    return a;
}
```

H01_05_oklid_ebob.cpp · satır 25–35 — C++

```cpp
int ebob(int a, int b)
{
    adim_sayisi = 0;
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
        adim_sayisi++;
    }
    return a;
}
```

H01_05_oklid_ebob.py · satır 21–28 — Python

```python
def ebob(a, b):
    """Öklid algoritmasıyla ebob(a, b)."""
    global adim_sayisi
    adim_sayisi = 0
    while b != 0:
        a, b = b, a % b
        adim_sayisi += 1
    return a
```

Python'un çoklu ataması `a, b = b, a % b` geçici değişkeni kaldırır

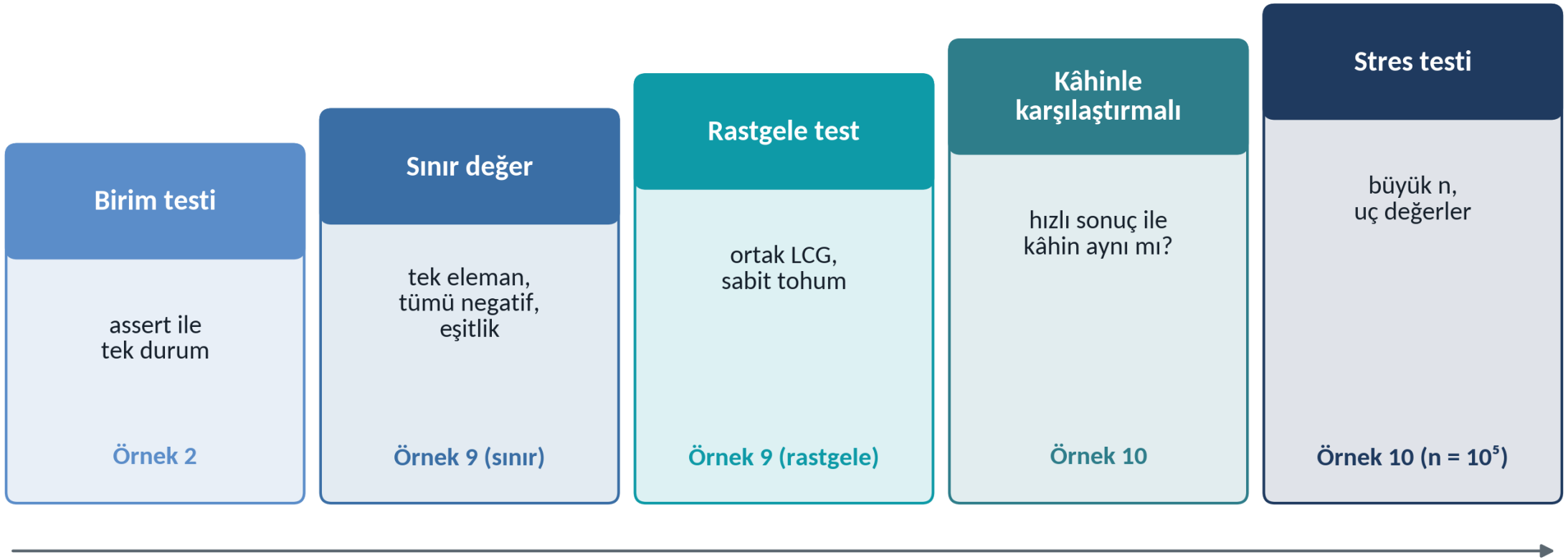
hardal satır = C'den farklılaşan satır

02

Test ve ölçüm

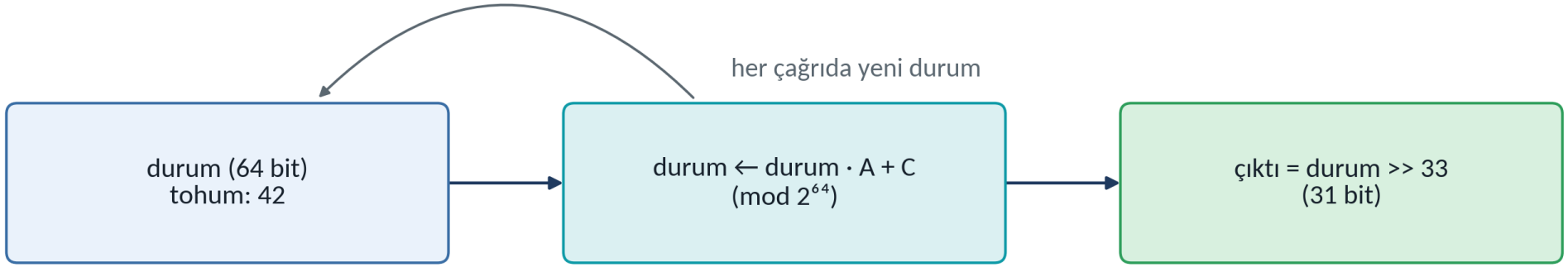
Kâhinle karşılaştırmalı test · ısınma, tekrar, ortanca · -O2

Test yelpazesi: birimden strese



Her basamak bu haftanın bir örneğiyle; kapsam ve maliyet sağa doğru artar

Tekrarlanabilir rastgele veri: rand() yerine ortak LCG

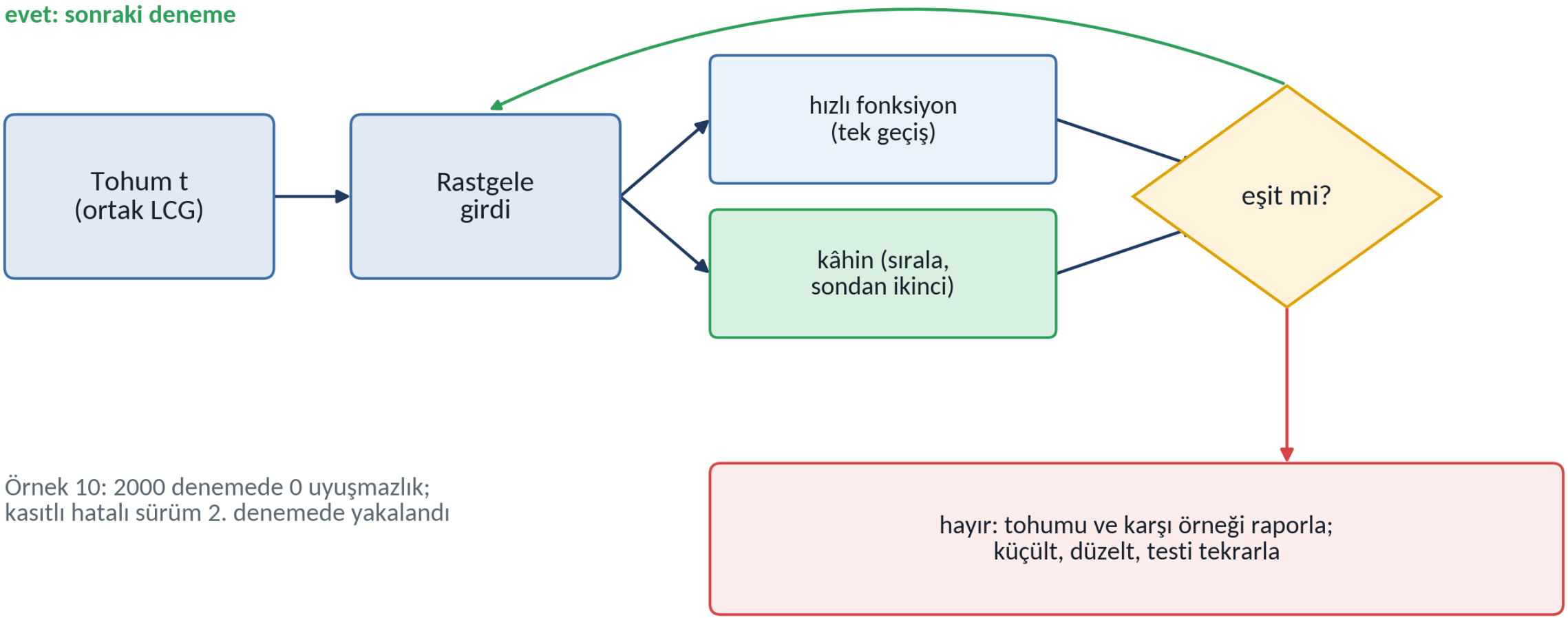



```
tohum 42, 1. adım:  durum = 10481999410520546993  →  çıktı = 1220265334
A = 6364136223846793005   C = 1442695040888963407
```

C/C++: uint64_t kendiliğinden sarar · Python: & (2^{64} − 1) maskesi · üç dilde aynı dizi

- **HA6 · `rand()` hatası:** standart algoritmayı belirlemez (yalnız `RAND_MAX` ≥ 32767); `srand(time(NULL))` ile iki çalıştırmanın toplamı 4944984 ve 4947312
- **Ortak LCG + sabit tohum:** üç dilde bayt bayt aynı dizi (Örnek 8); tohum deney kaydına ve CSV'ye yazılır

Kâhinle karşılaştırmalı test



Örnek 10: 2000 denemede 0 uyuşmazlık;
kasıtlı hatalı sürüm 2. denemede yakalandı

Aynı rastgele girdi hızlı fonksiyona ve kâhine verilir; uyuşmazlıkta tohum ve karşı örnek raporlanır

KOD

Testi sına: kasıtlı hatalı sürüm aynı testte

H01_10_ikinci_en_buyuk.c · satır 108–121

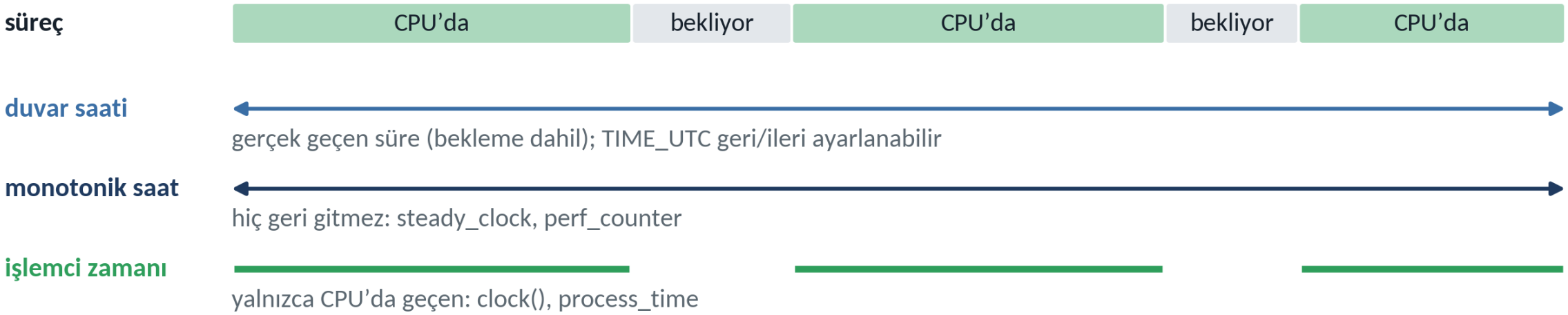
C

```c
    for (int t = 1; t <= deneme; t++) {
        size_t n = (size_t)rastgele_aralik(2, MAKS_N);
        rastgele_dizi(a, n, alt, ust);
        int beklenen = kahin(a, n);
        int bulunan = f(a, n);
        if (beklenen != bulunan) {
            uyusmazlik++;
            if (uyusmazlik == 1) {
                printf("  ilk uyuşmazlık (deneme %d): ", t);
                dizi_yaz(a, n);
                printf(" -> kâhin %d, bulunan %d\n", beklenen, bulunan);
            }
        }
    }
```

ÇIKTI

```
$ ./H01_10_ikinci_en_buyuk
3) Testi sına: kasıtlı hatalı sürüm aynı testte
  ilk uyuşmazlık (deneme 2): [4, -4, 1, 1, 2, -1, -4] -> kâhin 2, bulunan -4
  ikinci_en_buyuk_hatali, değerler [-5, 5]: 2000 deneme, 978 uyuşmazlık
```

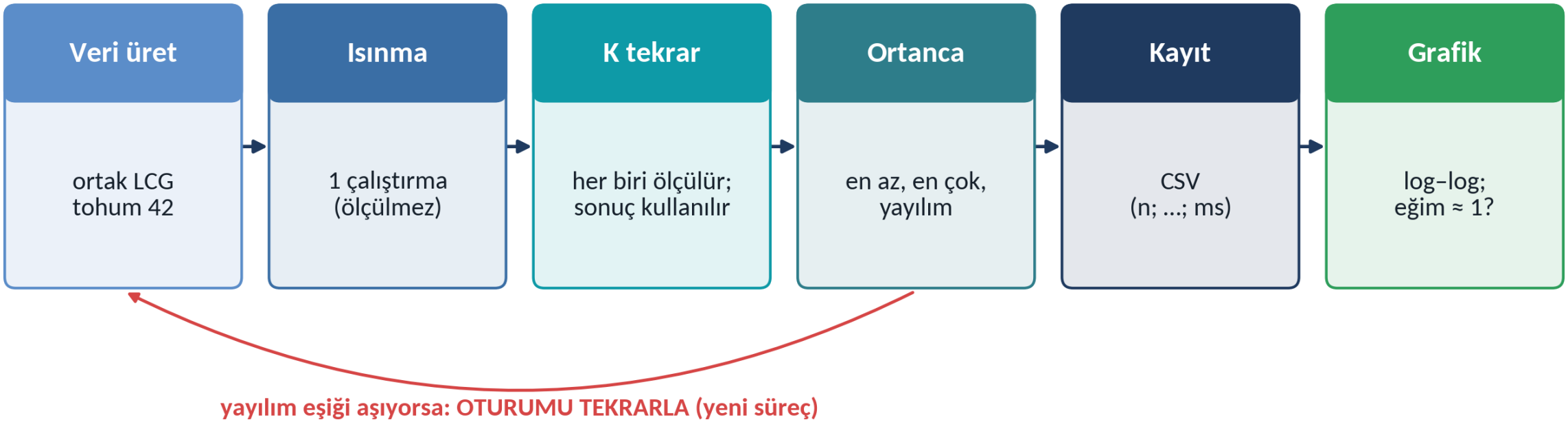
Kaputun Altında: ölçülen süre nereden gelir?



Çözünürlük: 1970'ten beri ≈ $1{,}76 \cdot 10^9$ s'yi double'da tutmak adımı 238,4 ns yapar;
zaman_sn() bu yüzden ilk çağrıya göreli süre döndürür (Kavram 4)

Süre için monotonik saat; işlemci zamanı bekleyen süreyi saymaz; tek ölçüm işletim sistemi gürültüsüne açıktır

Ölçüm düzeni: ısınma, tekrar, ortanca



süreler stderr'e / CSV'ye · stdout'a yalnızca deterministik sonuçlar · -O2 (karşılaştırma için -O0)

Tek ölçüme güvenilmez; yayılım büyükse oturumu (ayrı süreç) tekrarlayın (Sedgewick & Wayne, 2011)

Kaputun Altında: -O2 sonucu kullanılmayan döngüyü siler

kullanilmayan_dongu (-O2)

```
endbr64
ret
```

döngü YOK:
derleyici sildi

kullanilan_dongu (-O2)

```
endbr64
movq 8(%rdi), %rdx
testq %rdx, %rdx
je .L6
movq (%rdi), %rax
leaq (%rax,%rdx,4), %rsi
xorl %edx, %edx
.L5:
movslq (%rax), %rcx
addq $4, %rax
addq %rcx, %rdx
cmpq %rsi, %rax
jne .L5
movq %rdx, sonuc_yeri(%rip)
ret
…  (.L6: n = 0 dalı)
```

gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0 -O2 · sonuç yazılınca (sonuc_yeri) döngü kalır: 5 komutluk iç döngü

Kaputun Altında: üç maliyet modeli

C -O2: makine kodu (5 komut / eleman)

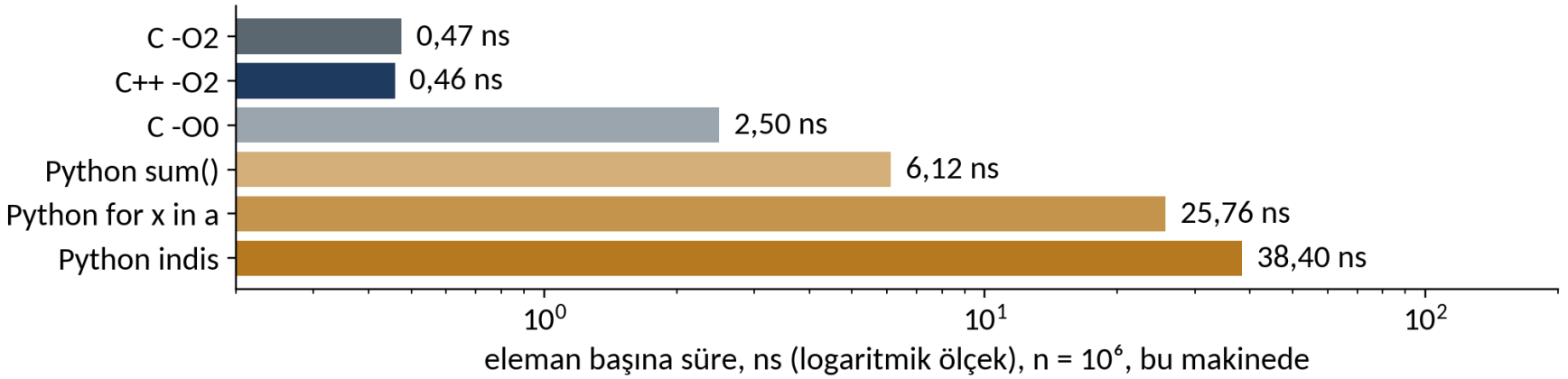
```
movslq  addq  addq
cmpq  jne
```

işlemci doğrudan yürütür

CPython: bayt kodu (9 komut / eleman)

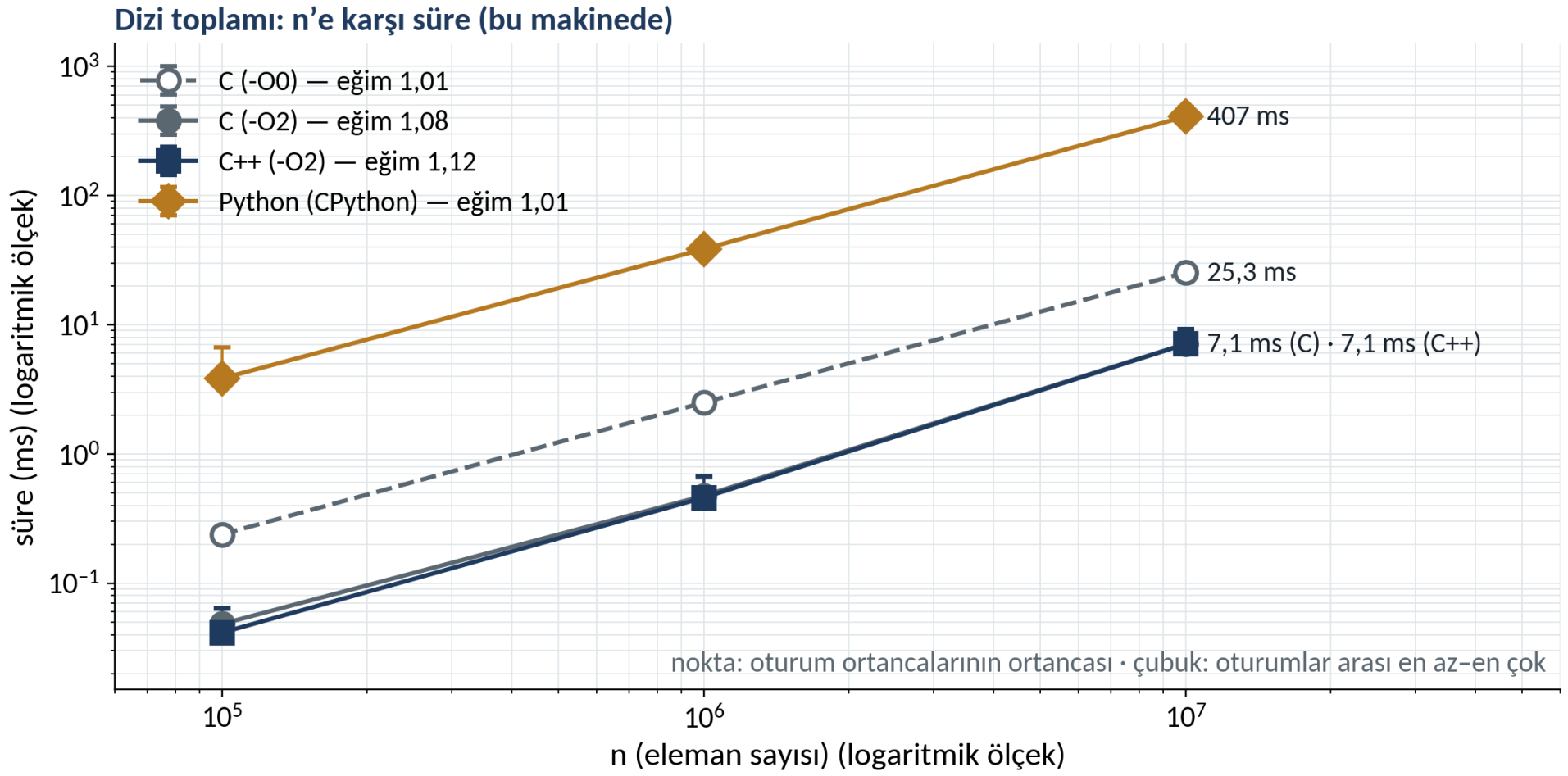
```
FOR_ITER  STORE_FAST  LOAD_FAST
LOAD_FAST  LOAD_FAST  BINARY_SUBSCR
BINARY_OP  STORE_FAST  JUMP_BACKWARD
```

yorumlayıcı döngüsü her komutu çözer; nesne işlemleri



Aynı döngü: 5 makine komutu ile 9 bayt kodu komutu; altta eleman başına ölçülen süre

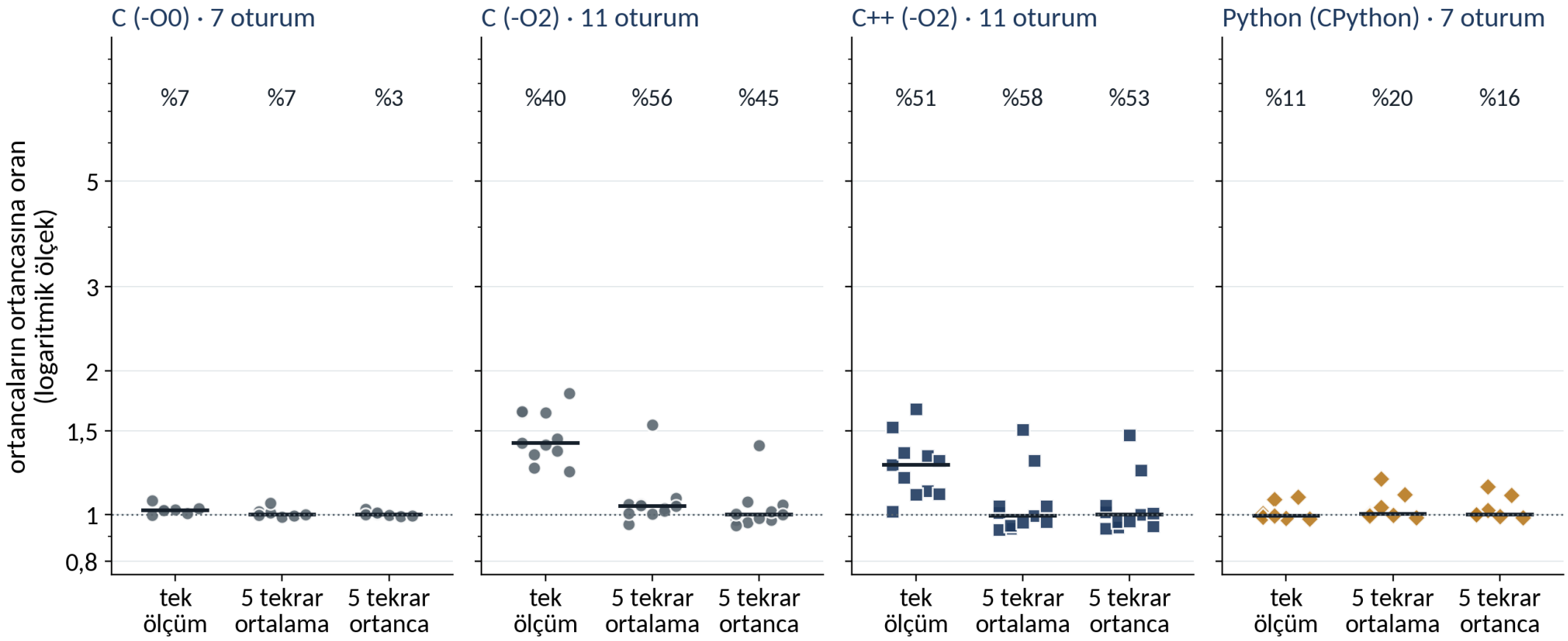
Haftanın Deneyi: aynı algoritma, üç dil, iki derleme düzeyi



log–log eğim: C -O0 1,01 · C -O2 1,08 · C++ -O2 1,12 · Python 1,01 (kuram: 1)

Tek ölçüm, 5 tekrarın ortalaması, 5 tekrarın ortancası

n = 10^6: oturum başına üç yöntem (nokta = bir oturum, çizgi = ortanca, üstte yayılım)



Oturumlar arası yayılım (en çok – en az) / ortanca; ısınmasız tek ölçüm sistematik olarak yüksek

Hata Avcısı 1: m = 0 ile başlatmak

Hatalı: m = 0, i = 0'dan

hata_avcisi/H01_HA1_en_buyuk_sifir.c · satır 12–16 — C

```c
int en_buyuk(const int a[], size_t n)
{
    int m = 0;                  /* HATA: 0 dizinin bir elemanı değil */
    for (size_t i = 0; i < n; i++) {
        if (a[i] > m) {
```

Düzeltilmiş: m = a[0], i = 1'den

hata_avcisi/H01_HA1_en_buyuk_sifir_duzeltilmis.c · satır 13–17 — C

```c
int en_buyuk(const int a[], size_t n)
{
    int m = a[0];               /* DÜZELTME: ilk elemanla başla */
    for (size_t i = 1; i < n; i++) {
        if (a[i] > m) {
```

Gerçek çıktı (−8, −3, −12, −5 °C) · hatalı: "Sıcaklıklar : en yüksek 0 °C" · düzeltilmiş: "Sıcaklıklar : en yüksek -3 °C". Puanlarda ikisi de doğru ("Puanlar : en yüksek 72"): tipik test hatayı göstermez; **tamamı negatif** dizi gösterir.

Hata Avcısı 4: işaretli taşma tanımsız davranıştır

Tanımsız: önce topla, sonra bak

hata_avcisi/H01_HA4_isaretli_tasma.c · satır 22–26 — C

```c
int tasar_mi_2(int x)
{
    int y = x + 1;              /* HATA: x == INT_MAX ise tanımsız */
    return y < x;
}
```

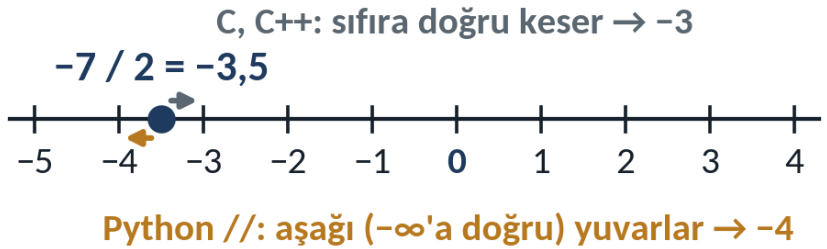
Doğru: işlemden önce sınırla karşılaştır

hata_avcisi/H01_HA4_isaretli_tasma_duzeltilmis.c · satır 20–23 — C

```c
int toplam_tasar_mi(int x, int y)
{
    return (y > 0 && x > INT_MAX - y) || (y < 0 && x < INT_MIN - y);
}
```

Bu derleyicide (gcc 13.3, x86-64) ve bu çalıştırmada gözlenen, x = INT_MAX: -O0 “2. sürüm: x + 1 taşar mı? evet” · -O2 “2. sürüm: x + 1 taşar mı? hayır” · UBSan: “signed integer overflow”. Başka derleyici ya da bayrakla farklı olabilir: davranış **tanımsızdır**.

Dil Penceresi: aynı görünen işlem, farklı anlam



C: $-7 = (-3)\cdot 2 + (-1)$

Python: $-7 = (-4)\cdot 2 + 1$

Öğe	C ve C++	Python
Tamsayı bölmesi	-7 / 2 → -3 (sıfıra doğru; C99, C++11)	-7 // 2 → -4 (aşağı yuvarlar)
Kalan	-7 % 2 → -1 (bölünenin işareti)	-7 % 2 → 1 (bölenin işareti)
Tek mi testi	x % 2 != 0 (x % 2 == 1 negatifte yanlış)	x % 2 == 1 de çalışır
Tamsayı genişliği	uint32_t 2^{32} modunda sarar (13! → 1932053504); işaretli taşma tanımsız	int keyfi duyarlıklı: 13! = 6227020800
assert	<assert.h> / <cassert>; -DNDEBUG kapatır	assert deyimi; python3 -O kapatır
Süre ölçümü	timespec_get, clock(); C++: steady_clock	time.perf_counter, process_time

- Üç dilde de (a / b)·b + a % b = a
- Gerçek çıktı: C “-3 -1”, Python “-4 1”

MÜHENDİS GÖZÜYLE

Test doğruluğu kanıtlamaz: kâhin, değişmez ve ölçüm birlikte

- **Değişmez + sonlanma:** fikir her girdide doğru mu?
- **Kâhinle test:** kod fikri doğru gerçekleştiriyor mu? (denenen girdilerde)
- **Ölçüm:** süre kuramla uyumlu mu; sabit çarpanlar ne?
- “Bitti” listesi: sözleşme + assert · değişmez · sınır testleri · kâhin + kayıtlı tohum · testin kasıtlı hatayla sınanması · -O2 + ısınma + ortanca
- Deney kaydı: veri (tohum, n), ortam (CPU, önbellek, yük), araç sürümleri, bayraklar, düzen, tarih

Merak Kutusu: sonlandığı kanıtlanamamış bir döngü

kavram/H01_K03_collatz.c · satır 15–27

C

```c
long collatz_adim(unsigned long long n, unsigned long long *tepe)
{
    long adim = 0;
    *tepe = n;
    while (n != 1) {
        n = (n % 2 == 0) ? n / 2 : 3 * n + 1;
        if (n > *tepe) {
            *tepe = n;
        }
        adim++;
    }
    return adim;
}
```

- n çiftse n/2, tekse 3n + 1
- Her n için sonlandığı **kanıtlanmış değil**
- Kesin azalan nicelik bilinmiyor
- 27 → 1: **111 adım**, tepe 9 232

MİNİ-OBS 2 PROJESİ

Mini-OBS 2 · v2.0: veri üreteci, ölçüm düzeni ve kâhinle test

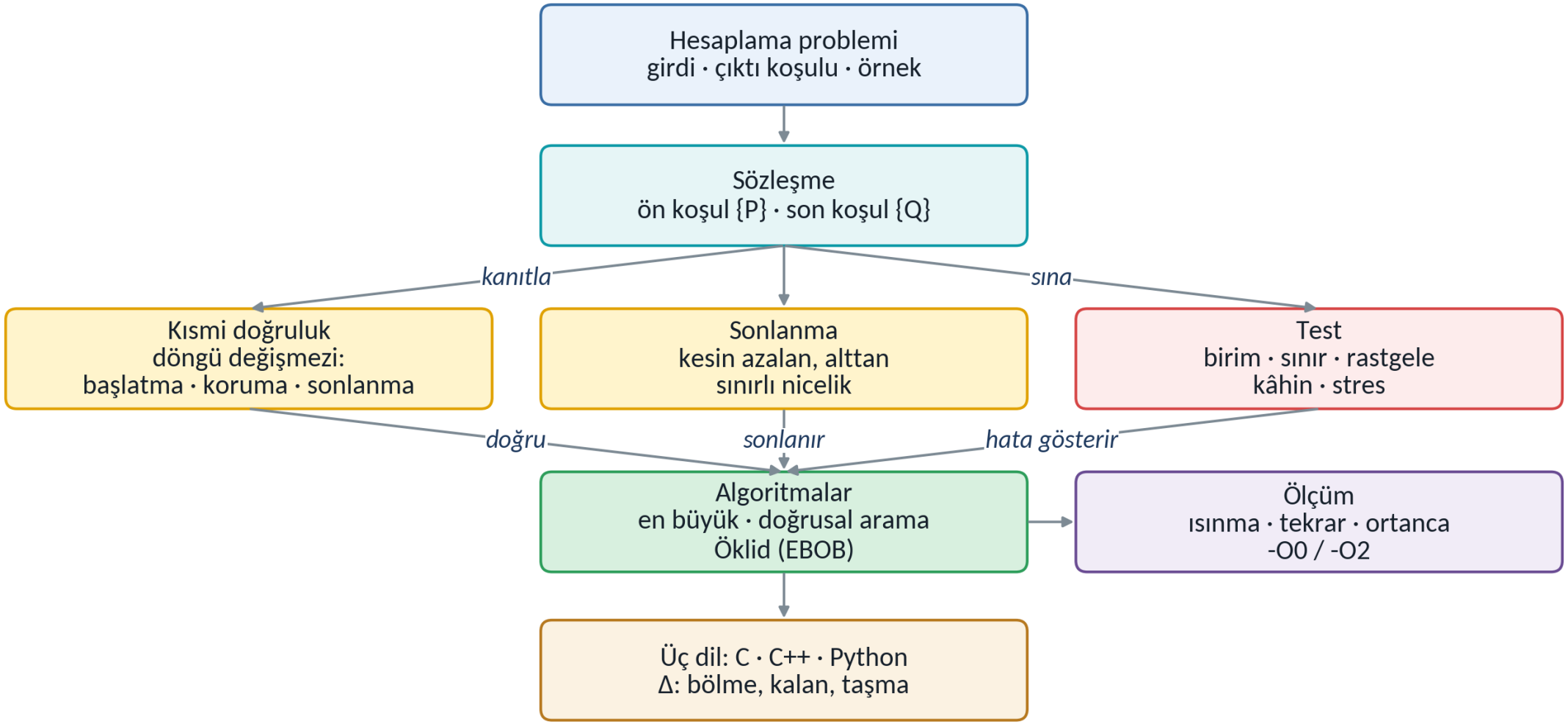
- **veri_uret:** ortak LCG ile 10^3–10^6 kayıt, tohumla bayt bayt tekrarlanabilir; `--yinelenen K`
- **deney + olcer:** 1 ısınma + K tekrar, ortanca; stdout deterministik, süre stderr/CSV
- **test_kahin:** en az üç istatistik fonksiyonu kâhinle; uyuşmazlıkta tohum ve karşı örnek
- **Bulgu:** v1.0 yükleyicisi karesel: 1 000 kayıt 1,5 ms, 100 000 kayıt 17,1 s (ortanca)
- Bu hafta yükleyici değiştirilmez: H2'de oran testi; H4 (v2.3) ve H6'da (v2.5) iyileştirme; H10 (v2.9) yinelenen tespitini O(n)'e indirir

İLERİ BAKIŞ

Bu haftanın araçları nerede derinleşiyor?

- **Değişmezler:** H4 ikili arama (aranan a[lo..hi] içinde), H5 araya ekleme (a[0..i−1] sıralı), H6 bölümleme, H12 Dijkstra
- **Kâhinle test:** her yeni algoritmanın kabul ölçütü; her sıralama kütüphaneyle karşılaştırılacak
- **Ölçüm ve oran testi:** H2'de T(2n)/T(n) ile büyüme hızını kestirmek
- **Biçimsel doğrulama:** Hoare mantığı, KeY, Frama-C: değişmezi makine denetler

Özet: haftanın kavram haritası



ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 `negatif_say` (Atölye 1, Kart 1) döngüsünün değişmezini yazın: döngü başında c neyi sayar?

2 Öklid algoritmasının sonlandığını hangi nicelikle gösterirsiniz? Neden kesin azalır ve alttan sınırlıdır?

3 C'de `-7 / 2` ve Python'da `-7 // 2` kaç verir? İkisinin de doğru olduğu özdeşlik hangisidir?

Gelecek haftaya hazırlık

- **Okuma:** ders notu Bölüm 3–4 ve Ek A; web sayfasındaki değişmez görselleştiricisi ve ölçüm laboratuvarı
- **Alıştırmalar:** 1.1–1.13 (F'de ★★★: 1.12 aynı geçişte en küçük ve en büyük)
- **Atölyeler (ders notu 11.2):** 1 "Değişmezi bul" kartları · 2 hata avı: beş fonksiyon · 3 ölçüm deneyi: üç dil, -O0/-O2 (Örnek 13) · 4 kurulum: `python3 esdeger_test.py .`
- **Mini-OBS 2 v2.0:** `veri_uret`, `olcer`, `test_kahin`; kabul ölçütleri 1–9
- **H2 · Algoritma Analizi: İşlem Sayma, Asimptotik Gösterim ve Büyüme Hızları:** 10^6 elemanda 1 saniyede çalışan bir program 10^9 elemanda ne kadar sürer ve bu soru, programın hangi dilde yazıldığından neden daha önemlidir?